BAB IX

DOKUMENTASI RULES, STANDARD, & TIPE PENULISAN CODE

Konvensi Penamaan Naming Standard, Prepared Statements SQL, & Async Error Handling

Penyusun / Penulis:	Saya (Fullstack Developer & System Architect)
Entitas Perusahaan:	PT. Global Philsonic Technologies (GPT) — Unit Bisnis PT. Intersis Sejahtera Abadi (ISA)
Kawasan Operasional:	Batam Free Trade Zone (FTZ), Kepulauan Riau, Indonesia
Tanggal & Status:	Oktober 2026 • Dokumen Final Teknis (Monokrom Standar)

9.1 Konvensi Penamaan (Naming Conventions)

Untuk menjaga konsistensi, keterbacaan, dan kemudahan kolaborasi jangka panjang, saya menetapkan aturan penamaan baku di seluruh repositori:

- **Variabel & Fungsi JavaScript:** Menggunakan format `camelCase`. Nama fungsi harus menggunakan kata kerja deskriptif yang mencerminkan tugasnya.
  Contoh: `getJobVacancies`, `calculateTotalExperience`, `isTokenExpired`.
- **Komponen React (.jsx):** Menggunakan format `PascalCase` sesuai standar resmi ekosistem React.
  Contoh: `Navbar.jsx`, `JobCard.jsx`, `AutomationPage.jsx`, `RobotCanvas3D.jsx`.
- **Berkas Controller & Helper:** Menggunakan format `camelCase.js`.
  Contoh: `publicJobController.js`, `authMiddleware.js`, `fileCleanup.js`.
- **Kolom & Tabel Database MySQL:** Menggunakan format `snake_case` dengan huruf kecil seluruhnya.
  Contoh: `job_vacancies`, `category_id`, `employment_type`, `created_at`.
- **Variabel Lingkungan (Environment Variables):** Menggunakan format `SCREAMING_SNAKE_CASE`.
  Contoh: `PORT`, `DB_HOST`, `JWT_SECRET`, `NODE_ENV`.

9.2 Standar Penanganan Kesalahan (Error Handling Pattern)

Saya mewajibkan setiap operasi I/O asinkron (asynchronous database query atau file system access) dibungkus dalam blok kontrol `try...catch` dengan alur penanganan terpadu:

```
// Standar Penulisan Controller Asinkron:
exports.getJobVacancyById = async (req, res, next) => {
  try {
    const { id } = req.params;

    // 1. Eksekusi query menggunakan prepared statement aman
    const [rows] = await db.query(
      'SELECT * FROM job_vacancies WHERE id = ? AND status = "published"',
      [id]
    );

    // 2. Evaluasi keberadaan data
    if (rows.length === 0) {
      return res.status(404).json({
        success: false,
        message: 'Lowongan pekerjaan tidak ditemukan atau sudah ditutup.'
      });
    }

    // 3. Kembalikan data dengan status 200 OK
    return res.status(200).json({
      success: true,
      data: rows[0]
    });
  } catch (error) {
    // 4. Teruskan galat internal ke middleware terpusat
    next(error);
  }
};
```

9.3 Standar Sanitasi Data & Proteksi SQL Injection

Saya memberlakukan aturan mutlak bahwa **tidak ada parameter masukan dari pengguna yang boleh langsung digabungkan (string concatenation) ke dalam perintah SQL**. Seluruh query wajib menggunakan fitur *Prepared Statements* dengan placeholder tanda tanya (`?`) yang disediakan oleh pustaka `mysql2/promise`. Hal ini menjamin sistem 100% aman dari ancaman manipulasi SQL Injection.

9.4 Format Respons API Seragam (Standardized JSON Envelope)

Semua rute API wajib mengembalikan respons terbungkus (envelope) dengan properti boolean `success` di tingkat akar. Dengan aturan ini, komponen frontend dapat melakukan pengecekan kondisi secara elegan:

```
// Pada sisi Frontend:
const response = await api.get('/jobs');
if (response.data.success) {
  setJobList(response.data.data);
} else {
  displayNotification(response.data.message);
}
```